

PASSWORD STRENGTH CHECKER

Python Cybersecurity Project Instructions

Project 01 | Beginner Security Automation | Portfolio Ready

Purpose of this packet

Use this guide to build the project yourself. It explains what the program must accomplish, the Python concepts you need, a recommended build order, testing expectations, GitHub organization, and a professional technical report framework. It intentionally does not provide finished code or scoring answers.

Name: _____

Start Date: _____

Repository: _____

1. Project at a Glance

Build a Python command-line script that evaluates the strength of a user-submitted password using common security criteria: length, uppercase letters, lowercase letters, numbers, and special characters. The program should return a simple strength rating and suggest specific improvements when requirements are missing.

Item	Project Requirement
Project type	Python command-line cybersecurity utility
Primary focus	Data validation, password policy logic, basic regular expressions, and security-minded feedback
Required output	A simple strength classification such as Weak, Medium, or Strong, plus improvement suggestions
Core import	Python built-in regular expression module: re
External packages	None required for the basic version
Portfolio deliverables	Python script, README, testing evidence, and technical project report

What this project should demonstrate

- You can translate a security policy into program logic.
- You understand basic input validation and Boolean checks.
- You can use regex to detect character categories inside a string.
- You can classify results using conditions and a scoring policy.
- You can provide actionable feedback instead of only displaying a pass/fail result.
- You can test and document a small security tool professionally.

Your final deliverables

- [] A working .py script.
- [] A README.md explaining the project and how to run it.
- [] A documented set of test cases or terminal screenshots using fake/sample passwords only.
- [] A technical report summarizing the problem, approach, validation, findings, and recommendations.
- [] Optional: a small tests folder or automated test file after the basic version works.

Security note

Do not test with your real password, reuse a real credential, save real passwords in screenshots, or commit passwords to GitHub. Use invented test strings only.

2. Python Concepts You Need

You do not need advanced Python for this project. The goal is to combine a small set of fundamentals into a security-focused program.

Concept	Why you need it	Required?
Variables	Store the submitted password, check results, score, and strength classification.	Yes
Strings	A password is text data that must be inspected character-by-character or pattern-by-pattern.	Yes
input()	Accept a password from the user in the basic command-line version.	Yes
len()	Evaluate the password length against the policy you define.	Yes
Booleans	Represent whether each security requirement passed or failed.	Yes
if / elif / else	Apply scoring rules, classify strength, and decide which suggestions to display.	Yes
Lists	Useful for collecting multiple improvement suggestions before displaying them.	Recommended
Functions	Keep validation, scoring, suggestions, and program flow organized.	Recommended
Return values	Allow functions to pass results back to the rest of the program.	Recommended
Loops	Not required for the basic checker; useful for printing suggestions, repeated checks, or multiple attempts.	Optional
Regex	Detect uppercase, lowercase, numbers, and special characters using patterns.	Yes
try / except	Only needed if you add features that can produce input conversion errors.	Optional

Do you need a loop?

Not necessarily. A single password can be evaluated without a loop. Add a loop only if you intentionally want repeated checks, repeated prompts, or a clean way to process a list of suggestions. Do not add a loop just to make the project look more complex.

Do you need functions?

You can build the first draft without functions, but the portfolio version should ideally use at least one function. Functions make it easier to separate responsibilities and explain your design during an interview.

3. Imports and Basic Regex Requirements

Required import

Use Python's built-in `re` module. You do not need to install it with `pip`. The project brief specifically calls for basic Python regex, so regex should be used for at least some of the character-category checks.

What you need to learn from `re`

- How `re.search()` checks whether a pattern exists anywhere inside a string.
- How to represent an uppercase-letter pattern.
- How to represent a lowercase-letter pattern.
- How to represent a numeric-digit pattern.
- How to represent a special-character or non-alphanumeric pattern.
- How a regex result can be treated as a match/no-match condition.

Do not overlearn regex for this project

You only need enough regex to detect the required character categories. Full regex mastery, capture groups, lookarounds, and advanced substitutions are outside the scope of this beginner project.

Optional professional enhancement

After the basic version works, you may research Python's `getpass` module so a password is not visibly echoed in the terminal while the user types it. This is optional and should be treated as a version-two improvement, not a requirement for starting the project.

Imports checklist

- [] `re` is included and used.
- [] No unnecessary third-party packages are installed.
- [] Optional modules are added only after the required version works.

4. Functional Requirements

Your script is complete only when it performs each required behavior consistently. Decide your exact policy before you write the scoring logic.

Requirement	What the program must determine
Password input	Accept one password value for evaluation.
Length check	Determine whether the password satisfies the length rule defined in your policy.

Requirement	What the program must determine
Uppercase check	Determine whether at least one uppercase letter is present.
Lowercase check	Determine whether at least one lowercase letter is present.
Number check	Determine whether at least one numeric digit is present.
Special-character check	Determine whether at least one qualifying special character is present.
Score	Convert passed/failed criteria into a simple score using a policy you define.
Classification	Translate the score into a simple strength label such as Weak, Medium, or Strong.
Suggestions	Identify failed requirements and produce specific improvement guidance.
Output	Display the result clearly and consistently without exposing or storing real credentials.

Security policy decisions you must make yourself

- ☐ What counts as the minimum acceptable length?
- ☐ Will you also recognize a stronger recommended length?
- ☐ How many points is each criterion worth?
- ☐ What score range counts as Weak?
- ☐ What score range counts as Medium?
- ☐ What score range counts as Strong?
- ☐ What exactly counts as a special character?
- ☐ What should happen when the user submits an empty password?
- ☐ Will spaces be allowed, rejected, or simply evaluated as characters?

Why these decisions matter

This is the security-policy part of the project. There is not one universal scoring formula for a beginner checker. What matters is that you define your rules, implement them consistently, test them, and document them.

5. Recommended Script Architecture

Think about the script as a small pipeline. Each stage receives information, performs one responsibility, and passes a result forward.

INPUT -> VALIDATE CRITERIA -> SCORE -> CLASSIFY -> BUILD FEEDBACK -> DISPLAY RESULT

Suggested sections inside the Python file

- 1. Imports:** Place required built-in modules at the top of the file.
- 2. Policy constants or settings:** Define the password rules your script will use so they are easy to find and change.
- 3. Validation logic:** Evaluate the five required criteria and store each result.
- 4. Scoring logic:** Convert the validation results into a simple score.
- 5. Classification logic:** Map the score to Weak, Medium, or Strong.
- 6. Suggestion logic:** Create improvement messages only for requirements that were not met.
- 7. Main program flow:** Collect the password, call the necessary logic, and display the result.

Function planning worksheet

Possible responsibility	Input	Output / responsibility
Evaluate password criteria	Password string	Pass/fail results for each security rule
Calculate score	Validation results	Numeric or count-based score
Classify strength	Score	Weak / Medium / Strong label
Build suggestions	Validation results	Zero or more improvement messages
Main program	User interaction	Coordinates the full workflow and prints the final result

You do not have to use every function shown above. The purpose is to think in responsibilities rather than placing every line inside one long block.

6. Step-by-Step Build Plan

Build one working layer at a time. Test each layer before adding the next one. This makes debugging much easier.

Phase 1 - Create the project scaffold

- ☐ Create the project folder and Python file.
- ☐ Add a short file header/comment describing the project purpose.
- ☐ Add only the required import(s).
- ☐ Get one password value from the user using a fake test password.

Phase 2 - Build the five security checks

- ☐ Create the length check.
- ☐ Create the uppercase check.
- ☐ Create the lowercase check.
- ☐ Create the number check.
- ☐ Create the special-character check.
- ☐ Temporarily verify that each check produces a clear pass/fail result.

Phase 3 - Create your scoring policy

- ☐ Decide how each passed criterion contributes to the score.
- ☐ Decide the score ranges for Weak, Medium, and Strong.
- ☐ Document the policy in your notes before coding the classification.

Phase 4 - Build the strength classification

- ☐ Use conditions to translate the score into one strength label.
- ☐ Test values near each classification boundary so there are no gaps or overlaps.

Phase 5 - Build improvement suggestions

- ☐ Create a suggestion for each missing criterion.
- ☐ Store multiple suggestions when several requirements fail.
- ☐ Make sure a strong result does not display irrelevant warnings.

Phase 6 - Clean the output

- ☐ Display the strength result in a readable format.
- ☐ Display suggestions in a clear list when needed.
- ☐ Do not print extra debugging values in the final version unless they are useful to the user.

Phase 7 - Refactor

- ☐ Move logical responsibilities into functions if the first draft is too crowded.
- ☐ Use descriptive variable and function names.
- ☐ Remove duplicate logic and unnecessary code.
- ☐ Add comments only where the reasoning is not obvious from the code.

Phase 8 - Test and document

- ☐ Run all planned test cases with synthetic passwords.
- ☐ Record what passed, what failed, and what you corrected.
- ☐ Create the README.
- ☐ Complete the technical report at the end of this packet.

7. Testing Plan

Testing is part of the project, not something you do only if the code breaks. Use made-up passwords that intentionally exercise one or more rules.

Test case category	What you are testing	Record your expected behavior
Empty input	How the script handles no characters at all.	_____
Very short password	Length policy and related feedback.	_____
Lowercase only	Missing uppercase, number, and special-character handling.	_____
Letters only	Character case checks and missing numeric/symbol feedback.	_____

Test case category	What you are testing	Record your expected behavior
Letters + numbers	Whether the script correctly detects mixed categories.	_____
No special characters	Specific feedback for the missing symbol requirement.	_____
All required categories	Whether every required check can pass together.	_____
Long password	Behavior above the minimum/recommended length.	_____
Contains spaces	Your chosen policy for spaces.	_____
Boundary score	Exact points where the strength label changes.	_____

For every test, record

- ☐ Test case name or ID.
- ☐ Synthetic input used.
- ☐ Expected result based on your policy.
- ☐ Actual result.
- ☐ Pass or fail.
- ☐ If it failed, what you changed and whether the retest passed.

Never put a real credential in your test evidence

Treat screenshots, test logs, and GitHub history as potentially public. Use obviously fake strings created only for this project.

8. Definition of Done

Use this checklist before you consider the project complete.

- ☐ The program runs without a syntax error.
- ☐ The program accepts one password for evaluation.
- ☐ The length rule is evaluated.
- ☐ Uppercase letters are detected.
- ☐ Lowercase letters are detected.
- ☐ Numbers are detected.
- ☐ Special characters are detected.
- ☐ Each criterion can be represented as pass/fail.
- ☐ A score is calculated from your documented policy.
- ☐ The score maps to one clear strength classification.
- ☐ Missing criteria produce relevant improvement suggestions.
- ☐ Strong passwords do not receive contradictory improvement messages.

- [] The program handles empty or unusual input according to your policy.
- [] The code is organized and readable.
- [] The script does not save or log real passwords.
- [] At least several different test categories have been executed.
- [] The README is complete.
- [] The technical report is complete.
- [] No real credentials or sensitive information are committed to GitHub.

What you should be able to explain out loud

- What problem the tool solves.
- What a password policy is.
- Why length is part of your policy.
- How regex helps identify character categories.
- How your scoring system works.
- Why your classification thresholds are reasonable for this practice project.
- How the program knows what feedback to provide.
- What you tested and what you learned from testing.
- What you would improve in a future version.

9. Professional GitHub Structure

Yes - structure the project like a small professional deliverable. A clean repository shows that you can code, document, test, and communicate your work.

Suggested repository name

password-strength-checker

Suggested folder layout

```
password-strength-checker/
  README.md
  password_strength_checker.py
  docs/
    technical_report.pdf
    screenshots/
  tests/                      (optional)
    test_cases.md             (or an automated test file later)
  .gitignore                  (optional but professional)
  LICENSE                      (optional)
```

README sections

- [] Project Title
- [] Overview
- [] Problem / Security Need
- [] Project Objectives
- [] Features

- [] Security Criteria / Policy Summary
- [] Technologies Used
- [] How to Run
- [] Testing
- [] Security Considerations
- [] Project Structure
- [] Lessons Learned
- [] Future Improvements

GitHub hygiene

- Never commit real passwords, API keys, tokens, personal credentials, or sensitive screenshots.
- Use fake examples in documentation.
- Use clear file names and a consistent naming style.
- Commit meaningful milestones instead of one giant final upload when possible.
- Write commit messages that describe what changed, not vague messages such as "stuff" or "update."
- Keep the repository focused on this project.

Example commit milestones - not code

- Initialize project and README.
- Add password criteria validation.
- Add strength scoring and classification.
- Add improvement feedback.
- Add test documentation.
- Add technical report and final documentation cleanup.

10. Evidence to Save While You Work

Do not wait until the end to remember what happened. Save a small amount of evidence during the project so your README and report are based on what you actually did.

Evidence	What to capture	Do not capture
Project setup	Repository/folder structure and the project file names.	Personal file paths containing sensitive information.
Working run	Terminal output using a fake test password.	A real password or credential.
Testing	A few representative test cases and final pass/fail results.	Every single debugging attempt unless it adds value.
Issue resolved	A short note describing a bug/problem and how you corrected it.	Sensitive system information.
Final state	Clean repository, README, and final program output.	Unnecessary personal information.

Mini project log

Issue encountered: _____

What I thought was causing it: _____

What I checked: _____

What fixed it: _____

What I learned: _____

Additional Project Notes

Use this space for decisions, test observations, questions to research, or details you may want to include later in the README or technical report.

11. Technical Writing Practice - Professional Project Report

Simulated organization context

Use this as a fictional workplace scenario for your report: An organization wants a lightweight password-strength validation utility to help users identify weak password characteristics before creating or updating an account password. You were assigned to build and validate a Python proof-of-concept that evaluates a password against a defined security policy and provides improvement guidance. This is a portfolio simulation - do not present it as real employment or a real client engagement.

Complete this report only after the program is finished and tested. Write in professional, factual language and describe what you actually built and observed. Do not include source code in the main report unless a short excerpt is necessary to explain a technical point.

Report cover information

Report Title: Password Strength Validation Utility - Technical Project Report

Prepared for: [Fictional Organization / Internal Security Team]

Prepared by: [Your Name]

Date: [Completion Date]

Version: 1.0

Classification: Portfolio / Training Simulation

1. Executive Summary

Write one short paragraph (about 4-7 sentences). Summarize the security problem, what you developed, how you validated it, the overall result, and the main recommendation. A manager should understand the project without reading the rest of the report.

Your notes: _____

2. Background and Problem Statement

Explain the organizational issue the tool is intended to address. Describe why inconsistent or weak password practices create a security concern and why a simple validation utility was proposed. Keep the scenario realistic and avoid claiming the tool replaces enterprise identity controls.

Your notes: _____

3. Project Objective

State the primary objective in one clear paragraph. Identify what the script was required to evaluate and what kind of feedback it was expected to return.

Your notes: _____

4. Scope and Requirements

List what was included in the project and what was intentionally outside scope. Include the five password criteria, scoring/classification requirement, improvement suggestions, and command-line execution. If you did not build authentication, password storage, breach checking, or enterprise policy enforcement, say so.

- Included: _____
- Included: _____
- Out of scope: _____
- Out of scope: _____

5. Technical Approach

Describe the design at a high level. Explain the programming language, built-in modules, how validation checks were organized, how results were converted into a score, and how feedback was generated. Focus on the approach, not line-by-line code.

Technologies used: _____

Key design decisions: _____

6. Security Policy Implemented

Document the policy you actually chose. This section should make your checker reproducible: another reader should understand what criteria were evaluated and how the strength categories were assigned, without needing to inspect the source code.

Policy element	Your implemented rule
Length	_____
Uppercase	_____
Lowercase	_____
Numbers	_____
Special characters	_____
Weak classification	_____
Medium classification	_____
Strong classification	_____

7. Testing and Validation

Summarize how you tested the program. Do not paste a huge test log. Explain the categories of test cases, whether expected and actual results matched, and any defects you found and corrected.

Test category	Expected	Actual	Status
_____	_____	_____	Pass / Fail
_____	_____	_____	Pass / Fail
_____	_____	_____	Pass / Fail
_____	_____	_____	Pass / Fail
_____	_____	_____	Pass / Fail

8. Issue Encountered and Resolution

Choose one genuine problem you encountered while building or testing the script. Describe the symptom, your troubleshooting process, the root cause you identified, the correction, and the successful retest. This is excellent technical-writing and interview practice.

Observed issue: _____

Investigation: _____

Root cause: _____

Resolution: _____

Retest result: _____

9. Results and Findings

Summarize what the completed program successfully does based on your testing. Mention any important behavior you observed. Do not claim more security capability than the tool actually provides.

Your notes: _____

10. Security Considerations

Explain how you handled sensitive information during development and what security concerns remain. Consider whether passwords are stored, logged, displayed on screen, included in screenshots, or committed to version control.

- Real credentials used during testing? Yes / No
- Passwords stored by the script? Yes / No
- Passwords written to logs/files? Yes / No
- Only synthetic passwords included in GitHub evidence? Yes / No
- Other consideration: _____

11. Limitations

Describe what the checker does not do. This shows professional judgment. Examples of categories to consider include password reuse, breached-password databases, dictionary/common-password

detection, entropy estimation, context-specific password policies, centralized enforcement, and secure authentication storage. Only list limitations that are accurate for your version.

- Limitation 1: _____
- Limitation 2: _____
- Limitation 3: _____

12. Recommendations and Future Improvements

Recommend logical next steps. Separate changes that improve the learning project from controls an actual organization should use. Do not imply that a simple strength checker is a substitute for MFA, an identity provider, or enterprise password-policy enforcement.

- Recommendation 1: _____
- Recommendation 2: _____
- Recommendation 3: _____

13. Conclusion

Write one concise paragraph stating whether the project objective was met, what the testing demonstrated, and what the most important takeaway was. Avoid introducing brand-new technical details in the conclusion.

Your notes: _____

Appendix - Evidence

- ☐ Repository link
- ☐ Selected terminal screenshot(s) using synthetic passwords
- ☐ Test-case summary
- ☐ Optional project flow diagram
- ☐ Optional code excerpt only when needed to support a technical explanation

12. Technical Report Quality Checklist

Before you publish the report to GitHub, review it as if another security analyst or hiring manager will read it.

- ☐ The report clearly explains the problem before describing the code.
- ☐ The objective is specific and measurable.
- ☐ The implemented password policy is documented accurately.
- ☐ The report distinguishes requirements, implementation, testing, results, and recommendations.
- ☐ Testing statements are based on tests I actually performed.
- ☐ At least one troubleshooting example is explained clearly.
- ☐ No real passwords, credentials, tokens, or private information appear anywhere.
- ☐ The report does not exaggerate what the tool can do.
- ☐ Technical terms are used correctly and explained when necessary.
- ☐ Paragraphs are concise and professional.

- [] Screenshots have captions if included.
- [] The report uses consistent headings and formatting.
- [] Spelling and grammar have been reviewed.
- [] The final PDF is added under the docs folder in GitHub.

Recommended final portfolio package

Artifact	Purpose
password_strength_checker.py	Shows the Python implementation.
README.md	Explains the project quickly to a GitHub visitor.
docs/technical_report.pdf	Demonstrates professional technical writing.
docs/screenshots/	Provides selected visual evidence of testing and final behavior.
tests/ or test_cases.md	Shows that validation was planned and documented.

Project-completion rule

Do not write the report first and then make the project match the report. Build and test the program, save evidence, and then document what actually happened. That habit will make your technical writing much stronger.

END OF PROJECT INSTRUCTIONS